

Design Note

Two Worlds — Structured ERP + Unstructured Chat

The problem, as I see it

Every business I looked at keeps its truth in two places that never fully agree. The ERP has the official numbers — what's owed, what's ordered, what's paid — but it only knows what someone typed in, so it's always a little behind reality. The WhatsApp chat has the real story — a dispute, a delay, a machine breaking down — but no structure and no IDs. Neither side alone gives a useful answer, and giving one without the other is worse than giving neither.

So I built everything around one rule: chat can explain, qualify, or caveat a number — it can never produce one. Every figure in an answer has to trace back to an actual database row. Almost every decision below is really just me protecting that one rule. I've laid the rest of this out level by level, the same way the brief was structured, so it's easy to line up a question with my answer to it.

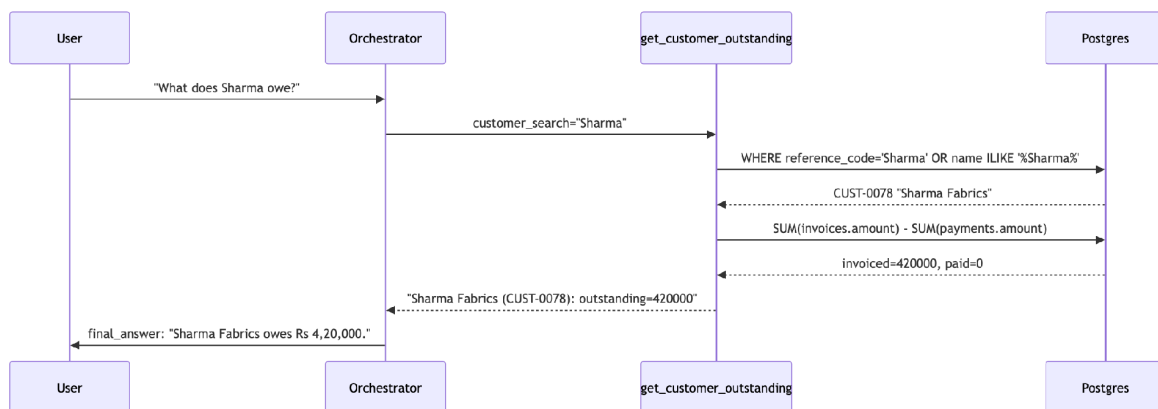
Level 1 — Two Worlds, Separately

Before combining anything, I wanted each side to work cleanly, and be trustworthy, on its own.

Structured querying

“the system can answer ‘what's Sharma's total outstanding?’ or ‘which POs are open past their promised date?’ from the tables”

Every question on the ERP side routes through a fixed SQL query — the same question gets the same query and the same answer every time, regardless of how it's phrased. Here's what that looks like end to end for “what does Sharma owe?”

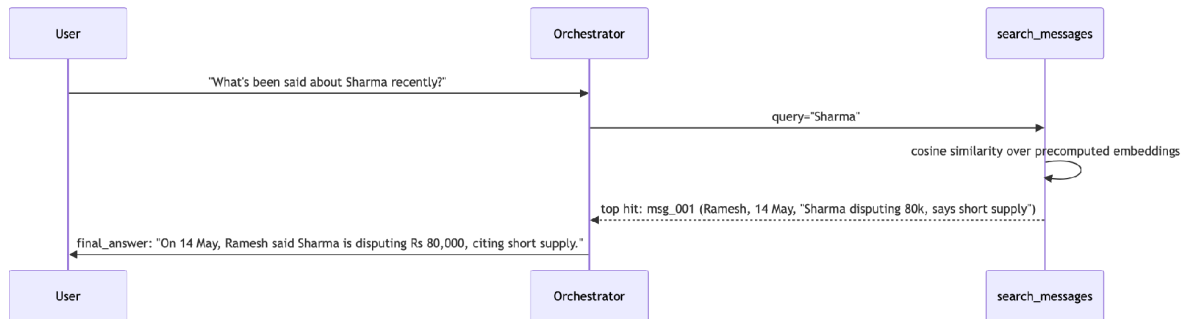


The structured path: a name resolves to a customer ID, then a fixed SQL aggregation, nothing left to interpretation.

Unstructured retrieval

“the system can surface relevant messages for ‘what's been said about Supplier B?’”

On the chat side, I turned every message into an embedding — a vector that captures what it means, not just the words it uses — so I can find the right message even when people phrase things completely differently (“supp B,” “the blue thread guys,” and “Supplier B” should all surface the same thread).



The unstructured path: similarity search over precomputed message embeddings, returned with sender and date attached.

Level 2 — Make Them Talk to Each Other

This is where the brief's own example questions live — “how much does Sharma owe, and is there anything I should know before I chase him?” — and where I spent most of my time. Three sub-problems, in the brief's own order.

2.1 — Entity linking across the boundary

“‘Supplier B’ in a WhatsApp message and SUP-1043 in the ERP are the same supplier ... a wrong link silently corrupts every number downstream. How do you build this bridge?”

This is the piece I spent the most time on, because a wrong link silently corrupts every number built on top of it. My approach is a small lookup table that maps informal phrases to exact record IDs, resolved in stages so a guess never quietly becomes a fact:

- Check if the phrase is already a trusted, human-confirmed mapping. If so, use it directly — no model involved.
- If not, score every candidate deterministically, using plain string similarity rather than an LLM call.
- Apply a threshold and a margin check: if the best match isn't clearly ahead of the runner-up, I mark it ambiguous instead of resolved. This is exactly what catches a trap like SKU-99201 vs. SKU-99210 — they look alike, so the margin collapses, so the system asks instead of guessing.
- The model's only role here is judging whether an ambiguous match is material enough to interrupt the user with a question. It never decides trust on its own.

A link only becomes trusted once a person confirms it. The same table also turns out to handle tenant-specific vocabulary — more on that under Level 3.4.

2.2 — Query planning

“Given a question, how does the system decide what to pull from where? Which parts need SQL, which need retrieval, which need both, and in what order?”

A request hits one endpoint, gets handed to the orchestrator, and enters a loop: call the model, see if it asked for a tool, run that tool, then check one thing — was the tool that just ran one of the two “response tools” (final_answer or ask_clarification), and did it succeed? If yes, the loop ends and that's the answer. If no, it goes back to the model with whatever the tool returned. The retrieval sub-agent runs the exact same loop, just with a narrower set of tools and its own response tool (return_claims), so there's only one piece of looping logic in the whole system deciding what to pull from where, not two.

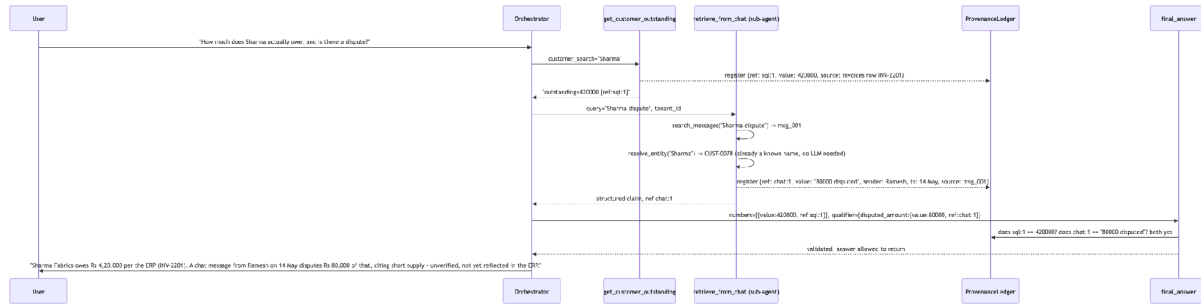
The tools themselves fall into a few groups — this is the actual inventory, not a simplified version of it:

Tool	Who calls it	What it returns
get_customer_outstanding, list_invoices, get_supplier_pos, get_late_orders, get_total_outstanding, get_total_po_quantity, list_customers, list_suppliers	Orchestrator	One fixed SQL query each — a customer's balance, a supplier's open POs, which orders are late, tenant-wide totals. Every number comes back tagged with a ledger reference; nothing here is a query the model gets to shape.
resolve_entity	Orchestrator & sub-agent	A trusted match, a scored list of candidates, or “ambiguous.” Never a silent guess dressed up as a match.
confirm_entity_link	Orchestrator	Writes a user-confirmed alias into the knowledge base. The only way a mention ever becomes trusted for next time.
search_messages	Sub-agent only	The top-k chat messages by meaning (cosine similarity over embeddings), each with its sender and date attached.
return_claims	Sub-agent only	Ends the sub-agent's turn with a list of structured claims — never a paragraph the orchestrator would have to trust blindly.
retrieve_from_chat	Orchestrator	Runs the entire sub-agent loop underneath, registers every claim it returns into the ledger, and hands back only the structured claims. This is the orchestrator's only window into chat — it never sees raw message text it could misquote.
final_answer	Orchestrator	Narrative + a list of numbers with refs. Rejected outright, before it can even run, if a ref is missing or a value doesn't match the ledger exactly.
ask_clarification	Orchestrator	Stops the turn and asks the user to pick between entity-link candidates, instead of guessing.

2.3 — The hard line

“Every number in an answer must come from the structured side ... Enforce this structurally, not by prompting.”

I didn't want to rely on a prompt telling the model “don't make up numbers,” because that's a request, not a guarantee. So every number a SQL tool returns gets registered in a shared ledger and handed back with a reference tag. When the model gives a final answer, it has to cite that tag — and before the answer is allowed out, the system checks the tag against the ledger and rejects the answer outright if the value doesn't match exactly. A fabricated or misquoted number simply can't pass; the model has to try again. Chat-derived amounts, like a disputed ₹80,000, get registered too, but only as something someone said — they're structurally blocked from entering the same field as a verified number.



The combined case: the SQL amount and the chat claim are each registered and validated independently before the final answer is allowed to go out.

Level 3 — Where It Gets Genuinely Hard

The brief's own instruction here was to go deep on a few of these rather than shallow across all six, so the honest status below is mixed on purpose — some of this is built and working, some is designed but not coded yet, and I've marked each one plainly rather than blurring the line.

3.1 — Contradiction

"The ERP says PO-4812 is open, full quantity, due Tuesday. A message from Friday says it was partially received, 300 short. Which does the system believe?"

Status: not built — the exact case is seeded, the comparison logic isn't.

"Latest wins" isn't good enough, since the ERP is authoritative but stale and chat is current but unverified. If I picked this up next, the design is to never resolve to one number: surface both facts with their timestamps, labeled by what each is good for — the ERP for anything needing certainty, chat for anything needing currency — rather than silently picking a winner.

3.2 — A number that changes the meaning of a number

"₹4,20,000 outstanding, but ₹80,000 is disputed per a message on 14 May. What's the right answer to 'how much does Sharma owe?'"

Status: partially built.

The important part already happens as a side effect of the hard line in 2.3: the disputed ₹80,000 can't slip into the verified numbers field, since it only carries a chat reference, not a SQL one. What's missing is a cleaner shape for the answer itself — a verified number with a clearly labeled qualifier next to it, instead of the caveat only living inside a sentence.

3.3 — Absence of evidence

"Nobody has mentioned Supplier C in chat for two months. Is that good news or missing data?"

Status: not built.

Right now, "no chat mentions" looks identical whether nothing's wrong or nobody's watching. The fix I'd want is a simple staleness check: if an entity has open orders and hasn't come up in chat past a set threshold, say that explicitly instead of staying silent about the silence.

3.4 — Tenant vocabulary

“One manufacturer says ‘lot’, another says ‘batch’, for the same thing ... How does tenant-specific vocabulary live in your system without forking code or retraining anything?”

Status: implemented.

This turned out to be the same mechanism as entity linking in 2.1, reused rather than rebuilt: one tenant's “lot” and another's “batch” are just two rows in the same lookup table, each pointing at the same item for their own tenant. Nothing in the code branches on which tenant is asking — a new tenant with its own words just needs its own rows.

3.5 — Provenance

“The owner asks ‘where did that come from?’ ... Design the data model for this from the start”

Status: implemented for numbers, partial for narrative.

Because everything is registered before the model ever sees it, that question always has a real answer: a SQL row for a number, a message with a sender and timestamp for a claim. I designed this in from the start rather than adding it later, since it's painful to retrofit. It's fully enforced for numbers today. For narrative claims, the reference already exists and is visible to the model, but the final answer doesn't yet require citing it the way it requires citing numbers — that's the one gap left in this piece.

3.6 — Evaluation

“How do you know the fused answers are right? ... Build something — even minimal — that regression-tests answer quality”

Status: not built.

I don't have a harness for this yet. I'd want 15–20 hand-picked cases — numeric answers checked directly against the database, narrative answers checked against expected message IDs — so a prompt or threshold change is something I can verify broke or didn't, instead of something I have to eyeball.

How I built the seed data and the chat search

A bit more detail on the engineering underneath the levels above, since it's easy to gloss over. I wanted the seed data to genuinely stress-test the system, not just look populated, so it's built in layers rather than randomly generated in one pass. First, I wipe and hand-write a small set of “anchor” records that mirror the brief's own examples almost exactly — a supplier, a customer called Sharma Fabrics, PO-4812 sitting at full quantity, an invoice for ₹4,20,000 with nothing paid. Then a seeded random generator fills in another 15–35 suppliers, customers, POs, and invoices around those anchors, so the tenant looks like a real, busy business and not three interesting rows in an empty database — “seeded” meaning I fixed the random seed, so re-running it produces the exact same data every time. Only after the ERP data exists do I generate the chat: a script reads back the real supplier and customer names from Postgres and drops them into WhatsApp-style templates, plus a handful of hand-written anchor messages that pair exactly with the ERP anchors — “Sharma disputing 80k, says short supply,” “4812 partially received, 300 units short.” That ordering (ERP first, chat second) is deliberate: it's what makes the chat chatter always reference entities that actually exist.

Turning that chat text into something searchable by meaning is the part that was genuinely new to me, so I want to walk through it properly rather than wave at it.

- Every message gets sent to an embedding model (OpenAI's text-embedding-3-small), which returns about 1,536 numbers per message — think of it as a fingerprint of what the message means, not what words it uses.
- Two messages about the same real thing, worded totally differently (“supp B running late” vs. “Supplier B pushing delivery back”), end up as two fingerprints that point in a similar direction in that 1,536-number space, even though barely a word overlaps.

- Cosine similarity is just a way of measuring how similar two directions are: scale every vector down to length 1 first (this is called normalizing), and then comparing two vectors becomes as simple as multiplying their matching numbers together and adding up the results. That one number lands close to 1 when the two messages mean something similar, and close to 0 when they don't — it genuinely is that simple once the vectors are normalized.
- Because normalizing turns the comparison into one multiplication, I can compare a new question against every stored message at once, as a single matrix multiplication, and just take the top few results. That's the whole search.
- Embeddings are computed once, up front, and saved — not recomputed on every search — because each call is a real, billed API call. A search only ever costs one embedding call: the question itself.

Tools and stack, and what's actually running today

Python, FastAPI, and PostgreSQL, as the brief asked for. For the agent loop itself I didn't reach for LangGraph in the end — I wrote a small state machine instead, because the one thing I needed to be absolutely sure of (the loop only ends when a response tool has actually succeeded, never because the model “decided” to stop talking) is something I wanted to point at in four functions of my own code, not trust to a framework's default behavior. Postgres access goes through psycopg behind one single function that's the only place in the codebase allowed to touch SQL directly; the domain objects are Pydantic models; OpenAI's text-embedding-3-small produces the message vectors; NumPy does the vector math described above.

What's actually built and working right now: the full agent loop end to end (orchestrator plus the retrieval sub-agent), every tool in the 2.2 table, the provenance ledger, entity linking with the ask-and-confirm loop, and the whole seed-data-to-embeddings pipeline. What isn't built yet is exactly what's marked as not built under Level 3 above.

Being honest about where I stand

Architecturally, I think this holds together — the trust rules, the ledger, and the reasoning behind each decision are things I understand and can walk through, and I'm reasonably happy with the shape of it. Where I'm less sure of myself is a layer underneath some of that. The vector search and embeddings piece especially — I got it working correctly by following the recipe (normalize, compare with a dot product, take the top matches), but not because I understood the math from first principles going in. That's not really how I'd want to work if this were headed to production, but this was a prototype meant to prove out the reasoning and the trust model, so I focused my time there and got the rest working as best I could. I'd genuinely like to go back and learn that piece properly rather than just knowing the recipe for it.

The code could also use a tidy-up pass in places — some of it grew organically while I was still working out the trust model. That's a smaller thing than the reasoning it's built on, but it's fair to mention.

What I'd do with another month

Contradiction handling (3.1) is where I'd start — it's the item that's most clearly a new reasoning capability rather than an extension of something already built, and the seed data is already sitting there ready for it. Evaluation (3.6) would come right after, since it's what would let me build the rest with an actual safety net instead of manual spot-checks. What I'm confident in regardless of time spent: the provenance ledger, the entity-linking trust rules, and the numbers-only enforcement — I built those first because everything else depends on them being solid.

One more note, for transparency: most of the code itself was written with the help of an AI coding agent. The schema, the entity-linking approach, the trust model, the provenance design, and the reasoning throughout this note are the result of my thought + LLM iteration — the agent helped turn that thinking into working code.